智能手环 MES 系统详细设计分析书

一、项目概述

1.1 项目背景

随着智能穿戴产业快速发展，智能手环生产呈现产品体积小、工艺精度高、型号迭代快、多批次小批量的特点。传统中小车间依赖人工记录、纸质单据的管理模式存在订单流转效率低、物料追溯困难、质量数据分散、异常处理滞后等痛点。本项目面向智能手环生产场景，开发一套轻量化制造执行系统（MES），通过数字化手段实现生产全流程管控，支撑车间降本提质，同时满足软件工程实训教学需求。

1.2 项目目标

1. **业务目标**：覆盖订单、采购、仓储、生产、质检、异常、设备、人员全流程数字化管理，实现物料一物一码追溯、生产进度实时监控、质量标准化管控、异常闭环处理。
2. **交付目标**：15 天内完成「网页管理端 + 小程序现场端」双端系统开发，核心功能可运行，代码规范可维护，配套完整开发文档。
3. **技术目标**：采用前后端分离架构，模块化设计，预留 ERP、设备接口，具备良好的可扩展性与可维护性。

1.3 业务范围与核心能力

系统共包含 9 大核心子系统，完整覆盖智能手环生产全链路：

子系统分类	核心业务能力
订单管理子系统	订单录入、审核、派工、状态跟踪、结案管理
采购管理子系统	采购计划制定、供应商管理、采购订单跟踪、来料登记
仓储物料管理子系统	物料入库赋码、出入库管理、生产备料发料、库存监控、退库核销

生产执行子系统	工单生成下发、物料领用、工序报工、进度跟踪、异常上报
工艺管理子系统	工艺路线设计、电子 SOP 编制、工艺参数配置
质量管理子系统	IQC 来料检验、工序巡检、成品抽检、不合格品管控、质量报告生成
设备管理子系统	设备台账、点检保养、故障维修、状态监控
人员组织管理子系统	员工信息、岗位权限、考勤排班、操作日志
数据统计与运维子系统	生产报表、质量分析、系统配置、数据备份

1.4 文档目的与读者对象

本文档为智能手环 MES 系统的详细设计依据，明确系统架构、模块划分、技术实现、开发分工与仓库规范，指导 4 人开发团队 15 天内完成项目落地。读者对象为开发工程师、测试人员与项目负责人。

二、系统总体设计

2.1 总体架构设计

系统采用**前后端分离 + 五层分层架构**，遵循高内聚低耦合原则，同时适配网页端与小程序端双入口。

2.1.1 技术分层架构

沿用业务建模中的分层设计，结合前后端分离模式优化为：

1. **前端展示层**：包含网页管理端（PC 浏览器）与小程序现场端（移动端），负责用户交互与数据展示。
2. **接口控制层（Controller）**：接收前端请求，完成参数校验、权限校验、请求分发与响应封装。

3. **业务服务层（Service）**：封装核心业务逻辑与规则，按业务域划分为 9 大模块，是系统核心层。

4. **数据访问层（DAO/Mapper）**：封装数据库增删改查操作，为业务层提供统一数据接口。

5. **数据存储层**：MySQL 8.0 结构化存储业务数据，支持事务与数据持久化。

2.1.2 端侧职能划分

端类型	使用角色	核心场景	特点
网页管理端	订单员、采购员、计划员、管理员、数据分析师	订单/采购/计划编制、数据统计、系统配置、报表查看	大屏操作、功能完整、复杂表单与数据可视化
小程序现场端	仓管员、生产操作员、质检员、异常处理员	扫码入库、领料报工、质检录入、异常上报、进度查询	移动端操作、扫码交互、轻量化功能、现场实时操作

2.2 技术选型

2.2.1 后端技术栈

技术组件	选型	说明
开发框架	Spring Boot 2.7.x	轻量级 Java 后端框架，快速搭建服务
ORM 框架	MyBatis-Plus 3.5.x	简化数据库操作，支持代码生成
数据库	MySQL 8.0	关系型数据库，符合项目部署要求
权限认证	JWT + Spring Security	实现多端统一鉴权、按岗授权
接口规范	RESTful API	统一接口设计规范

工具组件	Lombok、Hutool	简化代码、提升开发效率

2.2.2 网页管理端技术栈

技术组件	选型	说明
前端框架	Vue 3 + Vite	响应式前端框架，构建效率高
UI 组件库	Element Plus	企业级后台组件库，适配管理端场景
路由管理	Vue Router 4	页面路由与权限控制
状态管理	Pinia	全局状态管理
HTTP 请求	Axios	接口请求封装

2.2.3 小程序端技术栈

技术组件	选型	说明
开发框架	uni-app (Vue3 语法)	一套代码编译为微信小程序，兼顾开发效率
UI 组件库	uView Plus	移动端组件库，适配小程序场景
核心能力	微信扫码 API	实现物料/工单扫码操作
请求封装	uni-request	统一接口请求与异常处理

2.2.4 开发与部署工具

- 版本控制：Git + Gitee
- 开发工具：IDEA（后端）、VS Code（前端/小程序）、Navicat（数据库）
- 部署环境：Windows/Linux 服务器 + Nginx 反向代理

2.3 模块职责与端侧分配

子系统	网页管理端功能	小程序端功能	后端核心服务
订单管理	订单增删改查、审核、派工、状态跟踪、结案	订单进度查询	OrderService
采购管理	采购计划、供应商管理、采购订单跟踪	采购单查询、来料登记	PurchaseService
仓储管理	库存查询、出入库记录、库存盘点、台账管理	扫码入库、扫码出库、备料确认、物料追溯	StockService
生产执行	工单创建下发、工艺配置、进度监控、工单结案	接收工单、扫码领料、工序报工、进度上报	ProductionService
工艺管理	工艺路线设计、SOP 维护、工序配置	SOP 查看	ProcessService
质量管理	质检规则配置、检验记录查询、质量报表、不合格品跟踪	IQC 检验录入、工序巡检录入、成品抽检提交	QualityService
设备管理	设备台账、维保计划、故障记录、状态管理	设备点检、故障上报	DeviceService
人员管理	员工信息、角色权限、操作日志	个人信息、待办任务	UserService
数据统计	生产报表、质量分析、看板展示	个人产量统计	AnalyticsService

三、前端详细设计

3.1 网页管理端设计

3.1.1 页面结构与路由规划

采用「顶部导航 + 左侧菜单 + 主内容区」经典后台布局，路由层级不超过 3 级。

```
Plaintext
- 登录页 /login
- 首页 /dashboard（生产看板、核心数据概览）
- 订单管理 /order
  - 订单列表 /order/list
  - 订单详情 /order/detail/:id
  - 订单新增 /order/add
- 采购管理 /purchase
  - 采购单列表 /purchase/list
  - 供应商管理 /purchase/supplier
- 仓储管理 /stock
  - 库存查询 /stock/list
  - 出入库记录 /stock/record
  - 物料档案 /stock/material
- 生产管理 /production
  - 工单管理 /production/workorder
  - 进度监控 /production/monitor
  - 报工记录 /production/report
- 工艺管理 /process
  - 工艺路线 /process/route
  - SOP 管理 /process/sop
- 质量管理 /quality
  - IQC 检验记录 /quality/iqc
  - 工序质检 /quality/process
  - 质量报表 /quality/report
- 设备管理 /device
  - 设备台账 /device/list
  - 维保记录 /device/maintain
- 系统管理 /system
  - 用户管理 /system/user
  - 角色权限 /system/role
  - 操作日志 /system/log
```

3.1.2 核心页面交互规范

1. **列表页**：统一包含搜索栏、数据表格、分页组件、新增/编辑/删除操作按钮；批量操作需二次确认。
2. **表单页**：表单项按业务逻辑分组，必填项标记，提交前前端校验，提交后全局提示。
3. **详情页**：分模块展示核心信息、关联记录、操作日志，支持流程状态可视化。
4. **数据看板**：采用 ECharts 实现生产进度、质量合格率、异常统计等图表展示。

3.1.3 组件复用设计

封装通用业务组件，减少重复开发：

- 通用搜索表单组件
- 分页表格组件
- 订单状态标签组件
- 物料选择器组件
- 流程时间线组件

3.2 小程序现场端设计

3.2.1 页面结构与导航

采用底部 Tab 导航 + 页面级跳转的轻量化结构，核心页面聚焦现场操作，避免复杂交互。

```
Plaintext
底部 Tab：
1. 工作台（首页）
   - 待办任务（待检验、待报工、待处理异常）
   - 快捷操作入口
2. 扫码
   - 扫码识别物料/工单
   - 扫码结果页（信息展示 + 操作按钮）
3. 我的
   - 个人信息
   - 操作记录
   - 设置/退出登录
```

核心业务页面：
- 物料入库页
- 生产领料页
- 工序报工页
- IQC 检验录入页
- 异常上报页
- 工单详情页

3.2.2 核心操作流程设计

1. **扫码入库流程**：扫码获取物料批次 → 核对采购单与检验状态 → 确认入库 → 自动更新库存。

2. **工序报工流程**：扫码工单 → 选择当前工序 → 录入合格/不良数量 → 提交报工 → 更新工单进度。

3. **异常上报流程**：选择异常类型（物料/设备/质量） → 填写异常描述、上传照片 → 提交上报 → 自动推送通知。

3.2.3 硬件适配说明

- 调用微信小程序 `scanCode` API 实现条码/二维码扫描，兼容物料码、工单号扫码。
- 表单操作适配触屏交互，输入框优化数字键盘，关键操作按钮放大，适配车间现场操作场景。

四、后端详细设计

4.1 分层职责规范

4.1.1 控制层（Controller）

- 接收前端 HTTP 请求，进行参数基础校验与格式转换。
- 调用对应 Service 层方法处理业务，封装统一返回结果。
- 每个业务模块对应独立 Controller，按接口类型划分方法。
- 统一处理权限校验，通过注解实现接口级权限控制。

4.1.2 业务服务层（Service）

- 封装核心业务逻辑与业务规则，是系统业务核心。

- 按单一职责原则拆分，一个子系统对应一个主 Service，复杂逻辑拆分子 Service。
- 事务控制在 Service 层，涉及多表操作的业务方法添加事务注解。
- 业务规则校验（如未 IQC 检验物料不得入库、工单必须按工序顺序报工）在 Service 层实现。

4.1.3 数据访问层（Mapper/DAO）

- 基于 MyBatis-Plus 实现单表 CRUD，复杂查询编写自定义 SQL。
- 对应数据库表结构，一张表对应一个 Mapper 接口与 XML 文件。
- 禁止在 DAO 层编写业务逻辑，仅负责数据读写。

4.1.4 实体层

- **Entity**：数据库表映射实体，与表字段一一对应。
- **DTO**：前端请求数据传输对象，用于接收前端入参。
- **VO**：返回前端视图对象，用于封装返回数据。
- **BO**：业务内部对象，用于 Service 层间数据传递。

4.2 核心子系统业务设计

4.2.1 订单管理模块

- **核心流程**：订单录入 → 订单审核 → 生成生产工单 → 生产跟踪 → 订单结案。
- **业务规则**：审核驳回的订单需修改后重新提交；订单结案前必须核对生产数量与交付数量；紧急订单标记优先级并优先派工。
- **核心类**：Order、OrderItem、OrderService、OrderController。

4.2.2 仓储物料模块

- **核心流程**：来料接收 → IQC 检验 → 合格入库赋码 → 生产备料发料 → 生产退料 → 成品出库。
- **业务规则**：未 IQC 检验或不合格物料禁止入库；物料执行一物一码，无码物料不得领用；物料领用必须关联对应工单，禁止跨工单挪用。
- **核心类**：Material、Stock、StockRecord、StockService、StockController。

4.2.3 生产执行模块

- **核心流程**：生产工单创建 → 下发至工位 → 物料领用 → 工序作业 → 工序报工 → 成品检验 → 工单结案。
- **业务规则**：工单必须按预设工艺路线顺序执行，不得跳工序；关键工序强制关联质检，未质检不得进入下工序；报工数据需与实际进度一致，不得虚报。
- **核心类**：WorkOrder、ProcessReport、ProductionService、ProductionController。

4.2.4 质量管理模块

- **核心流程**：检验任务生成 → 检验数据录入 → 合格判定 → 生成检验报告 → 不合格品异常流转。
- **业务规则**：抽样质检遵循既定抽样比例；异常上报必须附完整不良描述；严重异常自动触发生产暂停；所有质检记录不可删除，仅可标注修改。
- **核心类**：IqcRecord、QcInspection、QualityService、QualityController。

4.3 设计模式应用

结合业务场景复用文档中的设计方案，提升系统可维护性：

1. **策略模式**：质检不合格判定（外观/功能/电池不同判定规则）、订单审核规则（普通/严格）。
2. **状态模式**：订单生命周期状态流转、质检单状态流转、工单执行状态管理。
3. **观察者模式**：异常上报后自动通知生产、质检、仓储等相关角色；订单状态变更同步关联模块。
4. **责任链模式**：异常分级处理流程，按异常等级匹配对应处理人。
5. **外观模式**：生产模块统一入口，封装工单、物料、异常子系统交互，简化上层调用。
6. **单例模式**：全局系统配置、质检规则参数唯一实例管理。

五、数据库详细设计

5.1 设计原则

1. 遵循第三范式，减少数据冗余，保证数据一致性。
2. 所有业务表包含创建人、创建时间、更新人、更新时间、逻辑删除标记，满足可追溯要求。
3. 关键字段建立索引，提升查询效率。

4. 命名统一采用下划线命名法，表名以 `biz_` 前缀区分业务表，`sys_` 前缀区分系统表。

5.2 核心数据表设计

一、系统管理类（4 张）

1. sys_user（系统用户表）

字段名	类型	主键	说明
user_id	bigint	是	用户唯一 ID
username	varchar(64)	否	登录账号，唯一
password	varchar(128)	否	加密存储的登录密码
real_name	varchar(32)	否	用户真实姓名
role_id	bigint	否	关联角色 ID
phone	varchar(16)	否	联系手机号
dept_name	varchar(64)	否	所属部门 / 车间
status	tinyint	否	账号状态：0 禁用 1 启用

2. sys_role（角色权限表）

字段名	类型	主键	说明
role_id	bigint	是	角色唯一 ID
role_name	varchar(32)	否	角色名称（如订单管理员、仓管员）
role_code	varchar(32)	否	角色编码，唯一标

			识
description	varchar(255)	否	角色职责描述
menu_permissions	text	否	菜单与接口权限 JSON 串
status	tinyint	否	角色状态：0 停用 1 启用

3. sys_operation_log（操作日志表）

字段名	类型	主键	说明
log_id	bigint	是	日志唯一 ID
user_id	bigint	否	操作人 ID
username	varchar(64)	否	操作人账号
module	varchar(64)	否	操作模块（如订单管理、仓储管理）
operation	varchar(128)	否	操作内容描述
request_method	varchar(16)	否	请求方式
request_params	text	否	请求参数
operation_ip	varchar(32)	否	操作 IP 地址
status	tinyint	否	操作状态：0 失败 1 成功
cost_time	int	否	请求耗时，单位毫秒
operation_time	datetime	否	操作时间

4. sys_dict_data（数据字典表）

字段名	类型	主键	说明
dict_id	bigint	是	字典 ID
dict_type	varchar(64)	否	字典类型（如 order_status、exception_type）
dict_label	varchar(64)	否	字典显示标签
dict_value	varchar(64)	否	字典存储键值
sort	int	否	排序号
status	tinyint	否	状态：0 停用 1 启用
remark	varchar(255)	否	备注说明

二、基础档案类（5 张）

5. biz_customer（客户档案表）

字段名	类型	主键	说明
customer_id	bigint	是	客户唯一 ID
customer_name	varchar(128)	否	客户名称
contact	varchar(32)	否	联系人姓名
contact_phone	varchar(16)	否	联系电话
address	varchar(255)	否	客户地址
credit_level	varchar(16)	否	信用等级

status	tinyint	否	合作状态：0 停用 1 正常

6. biz_supplier（供应商档案表）

字段名	类型	主键	说明
supplier_id	bigint	是	供应商唯一 ID
supplier_name	varchar(128)	否	供应商名称
contact	varchar(32)	否	对接联系人
contact_phone	varchar(16)	否	联系电话
address	varchar(255)	否	供应商地址
supply_category	varchar(64)	否	供应品类（如电子件、结构件）
qualification	varchar(255)	否	资质说明
status	tinyint	否	合作状态：0 停用 1 正常

7. biz_product（产品档案表）

字段名	类型	主键	说明
product_id	bigint	是	产品唯一 ID
product_code	varchar(64)	否	产品编码，唯一
product_name	varchar(64)	否	产品名称
product_model	varchar(64)	否	产品型号
specification	varchar(255)	否	规格参数描述

unit	varchar(16)	否	计量单位
default_route	text	否	默认工艺路线 JSON
status	tinyint	否	产品状态：0 下架 1 启用

8. biz_material（物料档案表）

字段名	类型	主键	说明
material_id	bigint	是	物料唯一 ID
material_code	varchar(64)	否	物料编码，唯一
material_name	varchar(64)	否	物料名称
material_type	varchar(32)	否	物料类型：电子件 / 结构件 / 包装件 / 辅料
specification	varchar(255)	否	规格型号
unit	varchar(16)	否	计量单位
default_supplier_id	bigint	否	默认供应商 ID
safety_stock	int	否	安全库存数量
status	tinyint	否	物料状态：0 停用 1 启用

9. biz_equipment（设备台账表）

字段名	类型	主键	说明
equipment_id	bigint	是	设备唯一 ID

equipment_code	varchar(64)	否	设备编号，唯一
equipment_name	varchar(64)	否	设备名称
equipment_model	varchar(64)	否	设备型号
workshop	varchar(64)	否	所属车间
location	varchar(64)	否	放置工位
buy_date	date	否	购置日期
maintain_cycle	int	否	保养周期，单位天
status	tinyint	否	设备状态：正常 / 维修中 / 停用

三、订单采购类（4 张）

10. biz_customer_order（客户订单表）

字段名	类型	主键	说明
order_id	bigint	是	订单唯一 ID
order_no	varchar(64)	否	订单编号，全局唯一
customer_id	bigint	否	关联客户 ID
total_quantity	int	否	订单总数量
delivery_date	date	否	约定交付日期
order_status	tinyint	否	订单状态：待审核 / 已审核 / 生产中 / 已完成 / 已结案

priority	tinyint	否	优先级：0 普通 1 紧急
order_remark	text	否	订单特殊要求
auditor_id	bigint	否	审核人 ID
audit_time	datetime	否	审核时间
close_time	datetime	否	结案时间

11. biz_order_item（订单明细表）

字段名	类型	主键	说明
item_id	bigint	是	明细 ID
order_id	bigint	否	关联订单 ID
product_id	bigint	否	关联产品 ID
quantity	int	否	产品数量
unit_price	decimal(10,2)	否	单价
remark	varchar(255)	否	明细备注

12. biz_purchase_order（采购订单表）

字段名	类型	主键	说明
purchase_id	bigint	是	采购单 ID
purchase_no	varchar(64)	否	采购单号，唯一
supplier_id	bigint	否	关联供应商 ID
related_order_id	bigint	否	关联销售订单 ID

purchase_type	varchar(32)	否	采购类型：正常生产 / 补料 / 备库
expect_arrive_date	date	否	预计到货日期
purchase_status	tinyint	否	采购状态：待确认 / 已发货 / 已到货 / 已完成
purchaser_id	bigint	否	采购员 ID
remark	varchar(255)	否	采购备注

13. biz_purchase_item（采购明细表）

字段名	类型	主键	说明
item_id	bigint	是	明细 ID
purchase_id	bigint	否	关联采购单 ID
material_id	bigint	否	关联物料 ID
purchase_quantity	int	否	采购数量
unit_price	decimal(10,2)	否	采购单价
arrive_quantity	int	否	已到货数量
remark	varchar(255)	否	明细备注

四、仓储库存类（3 张）

14. biz_warehouse（仓库库位表）

字段名	类型	主键	说明

warehouse_id	bigint	是	库位 ID
warehouse_code	varchar(64)	否	库位编码，唯一
warehouse_name	varchar(64)	否	库位名称
parent_id	bigint	否	父级 ID，支持多级仓库 - 库区 - 库位
warehouse_type	varchar(32)	否	类型：仓库 / 库区 / 库位
is_quarantine	tinyint	否	是否隔离区：0 否 1 是（存放不合格品）
status	tinyint	否	状态：0 停用 1 启用

15. biz_stock（库存台账表）

字段名	类型	主键	说明
stock_id	bigint	是	库存记录 ID
material_id	bigint	否	关联物料 ID
batch_no	varchar(64)	否	物料批次号
warehouse_id	bigint	否	所在库位 ID
quantity	int	否	当前库存数量
qc_status	tinyint	否	质检状态：待检 / 合格 / 不合格
barcode	varchar(128)	否	物料条码（一物一码）

expire_date	date	否	有效期
supplier_id	bigint	否	供应商 ID
inbound_time	datetime	否	入库时间

16. biz_stock_record（库存流水表）

字段名	类型	主键	说明
record_id	bigint	是	流水 ID
material_id	bigint	否	关联物料 ID
batch_no	varchar(64)	否	批次号
warehouse_id	bigint	否	库位 ID
record_type	varchar(32)	否	流水类型：入库 / 出库 / 调拨 / 盘点 / 退库
change_quantity	int	否	变动数量（入库为正，出库为负）
before_quantity	int	否	变动前库存
after_quantity	int	否	变动后库存
related_bill_no	varchar(64)	否	关联业务单据号
operator_id	bigint	否	操作人 ID
operate_time	datetime	否	操作时间

五、生产工艺类（2 张）

17. biz_work_order（生产工单表）

字段名	类型	主键	说明
work_order_id	bigint	是	工单 ID
work_order_no	varchar(64)	否	工单编号，唯一
order_id	bigint	否	关联客户订单 ID
product_id	bigint	否	关联产品 ID
process_route	text	否	工艺路线 JSON（工序顺序、参数）
plan_quantity	int	否	计划生产数量
finished_quantity	int	否	已完成合格数量
defective_quantity	int	否	累计不良数量
work_order_status	tinyint	否	工单状态：待下发 / 生产中 / 暂停 / 已完成 / 已结案
plan_start_time	datetime	否	计划开始时间
plan_end_time	datetime	否	计划结束时间
actual_start_time	datetime	否	实际开工时间
actual_end_time	datetime	否	实际完工时间
planner_id	bigint	否	计划员 ID
workshop	varchar(64)	否	生产车间

18. biz_process_report（工序报工表）

字段名	类型	主键	说明
report_id	bigint	是	报工 ID
work_order_id	bigint	否	关联工单 ID
process_code	varchar(32)	否	工序编码
process_name	varchar(64)	否	工序名称
operator_id	bigint	否	操作工 ID
equipment_id	bigint	否	使用设备 ID
qualified_quantity	int	否	本工序合格数量
defective_quantity	int	否	本工序不良数量
defect_type	varchar(64)	否	不良类型
is_pass	tinyint	否	是否过站：0 否 1 是
report_time	datetime	否	报工时间
remark	varchar(255)	否	报工备注

六、质量异常类（2 张）

19. biz_quality_inspection（质量检验记录表）

字段名	类型	主键	说明
inspection_id	bigint	是	检验记录 ID
inspection_no	varchar(64)	否	检验单号，唯一
inspection_type	varchar(32)	否	检验类型：IQC 来

			料 / IPQC 过程 / OQC 成品
related_bill_no	varchar(64)	否	关联业务单据号
material_id	bigint	否	关联物料 ID（来料检验用）
product_id	bigint	否	关联产品 ID（成品检验用）
batch_no	varchar(64)	否	批次号
sample_quantity	int	否	抽检数量
inspector_id	bigint	否	检验员 ID
check_result	tinyint	否	检验结果：0 不合格 1 合格
defect_items	text	否	不良项明细 JSON
check_time	datetime	否	检验完成时间
remark	text	否	检验备注

20. biz_exception_record（生产异常处理表）

字段名	类型	主键	说明
exception_id	bigint	是	异常 ID
exception_no	varchar(64)	否	异常编号，唯一
exception_type	varchar(32)	否	异常类型：物料 / 设备 / 质量 / 工艺
exception_level	tinyint	否	异常等级：1 轻微 2 一般 3 严重

work_order_id	bigint	否	关联工单 ID
reporter_id	bigint	否	上报人 ID
report_time	datetime	否	上报时间
description	text	否	异常描述与现场说明
handle_status	tinyint	否	处理状态：待处理 / 处理中 / 已闭环
handler_id	bigint	否	处理负责人 ID
handle_result	text	否	处理方案与结果
close_time	datetime	否	闭环时间

5.3 核心表关系说明

3. **订单驱动链路**：客户订单 → 订单明细 → 生产工单 → 工序报工，形成从订单到生产的全链路数据关联。

4. **采购仓储链路**：采购订单 → 采购明细 → 来料质检 → 库存台账 → 库存流水，实现物料从采购到入库的完整追溯。

5. **生产物料关联**：生产工单通过领料流水关联库存物料，实现物料消耗与工单的绑定，支撑物料批次追溯。

6. **质量闭环链路**：质检记录关联对应批次物料 / 产品，不合格项触发异常记录，形成质量问题从发现到闭环的完整链路。

- **人员权限关联**：用户关联单一角色，角色绑定菜单与接口权限，实现按岗授权、操作留痕。

六、接口设计规范

6.1 统一接口规范

1. **RESTful 风格**：GET 查询、POST 新增、PUT 修改、DELETE 删除。

2. **统一返回格式**：

```JSON
{
  "code": 200,
  "message": "操作成功",
  "data": {}
}
```

- 状态码：200 成功、400 参数错误、401 未授权、403 无权限、500 服务器错误。

3. **鉴权方式**：请求头携带 `Authorization: Bearer {token}`，后端解析 JWT 获取用户信息与权限。

4. **分页规范**：分页请求统一传 `pageNum`、`pageSize`，返回包含总条数与数据列表。

6.2 核心接口示例

接口地址	请求方式	接口说明	适用端
/api/auth/login	POST	用户登录	网页+小程序
/api/order/list	GET	订单列表查询	网页
/api/order	POST	新增订单	网页
/api/order/audit/{id}	PUT	订单审核	网页
/api/stock/scan/{barcode}	GET	扫码查询物料信息	小程序
/api/stock/inbound	POST	物料入库确认	小程序
/api/workorder/myList	GET	我的工单列表	小程序
/api/production/report	POST	工序报工提交	小程序
/api/quality/iqc/submit	POST	IQC 检验结果提交	小程序+网页

/api/exception/report	POST	异常上报	小程序

6.3 小程序端接口适配

- 小程序端接口路径统一加 `/mini` 前缀，与管理端接口区分，如 `/mini/production/report`。
- 小程序接口聚焦现场操作场景，简化入参，优化响应数据量，适配移动端网络环境。
- 所有接口支持 token 续期与异常统一处理，前端封装请求拦截器。

七、开发与部署方案

7.1 开发环境配置

环境	版本要求
JDK	1.8+
Node.js	16+
MySQL	8.0.x
Maven	3.6+
微信开发者工具	最新稳定版

7.2 部署架构

采用轻量化部署方案，适配中小车间单服务器部署：

1. **前端部署**：网页端打包后通过 Nginx 静态资源部署；小程序端提交微信审核发布。
2. **后端部署**：Spring Boot 项目打包为 Jar 包，部署在应用服务器，Nginx 反向代理。
3. **数据库部署**：MySQL 单实例部署，配置定时自动备份，对应文档中数据自动备份要求。
4. **网络配置**：服务器开放 80/443 端口，HTTPS 协议访问，保障数据传输安全。

7.3 测试方案

1. **单元测试**：后端核心 Service 方法编写单元测试，覆盖核心业务规则。
2. **功能测试**：按业务流程走测，验证订单→采购→入库→生产→质检→出库全链路闭环。
3. **兼容性测试**：网页端兼容 Chrome、Edge 主流浏览器；小程序兼容主流安卓、iOS 机型。
4. **异常测试**：验证参数错误、权限不足、网络异常等场景的系统容错能力。

八、15 天项目排期与 4 人分工

8.1 人员角色划分

成员	角色	核心职责
成员 A	后端负责人	架构搭建、核心模块开发、数据库设计、接口规范制定
成员 B	后端开发	业务模块开发、接口联调、单元测试、数据库脚本维护
成员 C	网页前端开发	管理端页面开发、组件封装、接口对接、页面优化
成员 D	小程序开发+测试	小程序端开发、测试用例编写、全流程测试、部署配合

8.2 整体开发排期（共 15 天）

阶段	时间	核心目标
第一阶段：环境搭建与基础设计	第 1-2 天	仓库搭建、项目骨架初始化、数据库设计完成、基础框架跑通

第二阶段：核心业务模块开发	第 3-9 天	完成订单、仓储、生产、质检核心功能开发，前后端并行
第三阶段：联调与补充功能	第 10-12 天	前后端接口联调、补充设备/人员/统计模块、修复开发 bug
第四阶段：测试优化与交付	第 13-15 天	全流程测试、bug 修复、性能优化、文档完善、项目交付

8.3 详细分工与日计划

第 1-2 天：初始化阶段

- **全员**：需求对齐、技术方案确认、Gitee 仓库拉取
- **成员 A**：后端项目骨架搭建、权限模块开发、数据库表结构设计与脚本生成
- **成员 B**：基础工具类封装、通用返回与异常处理搭建
- **成员 C**：网页端项目初始化、布局框架搭建、路由与权限基础配置
- **成员 D**：小程序项目初始化、底部导航搭建、登录页开发、请求封装

第 3-5 天：第一业务迭代（订单+采购+仓储）

- **成员 A**：订单管理、采购管理后端接口开发
- **成员 B**：仓储管理、物料档案后端接口开发
- **成员 C**：订单管理、采购管理页面开发与接口对接
- **成员 D**：小程序仓储扫码、入库/出库页面开发与接口对接

第 6-9 天：第二业务迭代（生产+质检+异常）

- **成员 A**：生产工单、工艺管理后端接口开发
- **成员 B**：质量管理、异常处理、设备管理后端接口开发
- **成员 C**：生产管理、质量管理、设备管理页面开发与对接
- **成员 D**：小程序工单列表、工序报工、质检录入、异常上报页面开发

第 10-12 天：联调与补充功能

- **成员 A**：数据统计接口、系统管理接口开发、核心流程联调
- **成员 B**：人员管理、操作日志接口开发、配合联调、单元测试补充
- **成员 C**：数据看板、系统管理页面开发、全页面联调优化
- **成员 D**：小程序个人中心、待办任务、联调 bug 修复

第 13-14 天：测试与修复

- **全员**：全流程走测、bug 提交与修复
- **成员 D**：编写测试用例、执行功能测试、输出 bug 清单
- **成员 A/B**：后端 bug 修复、性能优化、数据校验加固
- **成员 C/D**：前端交互优化、异常场景适配、样式统一

第 15 天：交付收尾

- **成员 A**：部署脚本编写、项目打包部署验证
- **成员 B**：数据库脚本整理、接口文档输出
- **成员 C**：网页端打包、使用说明整理
- **成员 D**：小程序打包、测试报告输出
- **全员**：代码提交合并、仓库整理、项目复盘

九、Gitee 仓库构建与协作规范

9.1 仓库创建与初始化

1. **仓库名称**：`smart-bracelet-mes`
2. **仓库简介**：智能手环生产制造执行系统（MES），包含网页管理端与小程序现场端。
3. **可见性**：私有仓库，添加 4 位开发成员为开发者权限。
4. **初始化文件**：自动生成 `.gitignore`、`README.md`、开源许可证。

9.2 仓库目录结构

```
Plaintext
smart-bracelet-mes/
```

```
├── backend/                    # 后端 SpringBoot 项目
│   ├── src/
│   ├── pom.xml
│   └── README.md
├── web-admin/                  # 网页管理端 Vue 项目
│   ├── src/
│   ├── package.json
│   └── README.md
├── mini-app/                   # 小程序 uni-app 项目
│   ├── src/
│   ├── package.json
│   └── README.md
├── docs/                       # 项目文档
│   ├── 需求文档.md
│   ├── 详细设计说明书.md
│   └── 接口文档.md
├── sql/                        # 数据库脚本
│   ├── schema.sql              # 表结构脚本
│   └── init_data.sql           # 初始化数据脚本
└── README.md                   # 项目总说明
```

9.3 分支管理策略

采用 Git Flow 简化版，适配小型团队快速开发：

1. **master 分支**：主分支，存放稳定可运行的代码，仅在版本交付时合并，禁止直接提交。
2. **dev 分支**：开发主分支，日常开发代码合并至此，保持最新开发进度。
3. **feature/xxx 分支**：功能分支，从 dev 拉取，用于开发单个功能模块，命名规范 `feature/模块名-功能名`，如 `feature/order-manage`。
4. **bugfix/xxx 分支**：bug 修复分支，从 dev 拉取，命名规范 `bugfix/问题描述`，如 `bugfix/stock-count-error`。

9.4 代码提交规范

采用 Conventional Commits 提交规范，提交信息格式：

```
Plaintext
<type>: <subject>
```

常用 type 类型：

- `feat`：新增功能
- `fix`：修复 bug
- `docs`：文档修改
- `style`：代码格式调整，不影响逻辑
- `refactor`：代码重构
- `test`：测试相关
- `chore`：构建工具、依赖配置修改

示例：

```
Plaintext
feat：新增订单列表查询接口
fix：修复物料入库库存计算错误
docs：更新数据库设计文档
```

9.5 协作流程

1. 每位开发者从 `dev` 分支拉取自己的功能分支进行开发。
2. 功能开发完成后，提交 Pull Request（PR）到 `dev` 分支，由后端/前端负责人审核后合并。
3. 每日下班前提交当日代码，保持分支同步，避免合并冲突。
4. 阶段开发完成后，统一在 `dev` 分支进行联调测试，测试通过后合并到 `master` 分支并打版本标签。
5. 重大问题及时沟通，禁止强制推送覆盖远程分支代码。